2.b Keywords, Imports & Visibility Modifiers

Understanding Kotlin Language Fundamentals

Contents

1 **Kotlin Keywords**
Hard keywords, soft keywords, modifier keywords & special identifiers

2 **Packages & Imports**
Package declarations, import directives, default imports & aliasing

3 **Visibility Modifiers**
public, private, protected, internal — scoping rules for declarations

4 **Practice Exercises**
Solved and unsolved exercises combining all three topics

What Are Keywords?

Keywords are predefined, reserved words in Kotlin that have special meanings to the compiler. They define the structure and behavior of your code.

Kotlin has 4 categories of keywords:

Hard Keywords

Cannot be used as identifiers.
Examples: val, var, fun, class,
if, else, when, for, while,
return, object, interface, null

Soft Keywords

Act as keywords only in context;
can be used as identifiers.
Examples: by, catch, constructor,
finally, get, set, import, where

Modifier Keywords

Keywords in modifier lists of
declarations; usable as identifiers.
Examples: public, private, internal,
open, abstract, override, data

Special Identifiers

Defined by the compiler in
specific contexts.
Examples: field (backing field),
it (lambda parameter)

Hard Keywords

Cannot be used as identifiers — strictly reserved

```kotlin
// Hard keywords example
fun main() {
    val name = "Kotlin"
    var count = 0
    if (count == 0) {
        println("Hello, $name!")
    } else {
        println("Count: $count")
    }
}
```

Key Points

- Kotlin has **28 hard keywords**
- They **cannot** be used as variable, function, or class names
- Include: `val, var, fun, class`
- Control flow: `if, else, when, for, while`
- OOP: `object, interface, this, super`
- Other: `return, throw, try, null, in, is, as`

Soft & Modifier Keywords

Soft Keywords — Context-dependent

```
// "by" is a soft keyword
val by = 10        // valid! Used as identifier
println(by + 10)   // prints 20

// "by" as keyword (delegation)
val lazyVal: String by lazy { "Hello" }
```

Soft Keywords:

- by, catch, constructor, delegate
- dynamic, field, file, finally, get, set
- import, init, param, property, where
- receiver, setparam, value
- they can be used as identifiers, when they are not in their keyword context – but avoid using them this way – it is chaotic

Modifier Keywords — Used in declarations

- **Visibility:** `public, private, protected, internal`
- **Inheritance:** `abstract, open, override, final, sealed`
- **Class modifiers:** `data, enum, inner, companion, annotation, value`
- **Function modifiers:** `inline, infix, operator, suspend, tailrec, external, vararg`
- **Other:** `const, lateinit, crossinline, noinline, actual, expect`

Packages in Kotlin

Package declaration goes at the **top of the source file** — directories and packages **don't need to match**

Package Declaration

```
package org.example.project

fun greet() = println("Hello!")
```

Default Imports (auto-imported)

```
kotlin.*                kotlin.collections.*
kotlin.annotation.*     kotlin.comparisons.*
kotlin.io.*             kotlin.ranges.*
kotlin.sequences.*      kotlin.text.*
// JVM also: java.lang.*  kotlin.jvm.*
```

Key Rules

- Source files can be placed **arbitrarily in the file system** (no directory-matching required)
- If no package is declared, the file belongs to the **default (nameless) package**
- Functions, properties, classes, objects, and interfaces can all be declared at **top-level** inside a package
- Several standard library packages are imported **automatically** into every Kotlin file

Import Directives

```
// 1. import a single name
import org.example.Message

// 2. Star import (all contents)
import org.example.*

// 3. Renaming import (alias)
import org.example.Message
import org.test.Message as TestMessage

// Also: functions, properties, objects
import org.example.greet
import org.example.MAX_COUNT
```

3 Types of Import

1. Single import

- Imports one specific declaration
- Accessible without qualification

2. Star import (*)

- Imports everything in the scope
- Lower priority in overload resolution

3. Renaming import (as)

- Resolves name clashes
- Only accessible by the new name

What can you import?

- Classes, top-level functions, properties, objects, enum constants

Visibility Modifiers

Kotlin has **4 visibility modifiers**: private, protected, internal, and public. The **default is public**.

public

Visible everywhere. This is the default if no modifier is specified.

private

Visible only inside the file (top-level) or the class (members).

protected

Like private, plus visible in subclasses. Not available for top-level declarations.

internal

Visible everywhere in the same module. A module = set of files compiled together.

Top-Level Declarations

- If you don't specify a modifier, **public** is used by default — visible everywhere
- **private** = visible only in that file | **internal** = visible in the same module
- **protected is NOT available** for top-level declarations

kotlinlang.org/docs/visibility-modifiers

Visibility in Classes

```
open class Outer {
    private val a = 1
    protected open val b = 2
    internal open val c = 3
    val d = 4  // public by default
}

class Subclass : Outer() {
    // a is NOT visible
    // b, c and d are visible
    override val b = 5  // still protected
    override val c = 7  // still internal
}

class Unrelated(o: Outer) {
    // o.a, o.b are NOT visible
    // o.c, o.d are visible (same module)
}
```

Class Members Rules

- `private` → visible only inside the class
- `protected` → class + subclasses
- `internal` → same module
- `public` → visible everywhere (default)

Important Rules

- Outer class does **NOT** see private members of inner classes
- Overriding a protected/internal member keeps its original visibility
- Constructor visibility requires explicit `constructor` keyword

Practice Exercises

Exercise 1: Package & Visibility

Create a file with a package declaration, a public function, and a private helper. Then import and use the public function from another file.

Solution

```kotlin
// File: Utils.kt
package com.school.utils

private fun formatName(name: String) =
    name.trim().uppercase()

fun greetUser(name: String): String {
    return "Hello, ${formatName(name)}"
}

// File: Main.kt
import com.school.utils.greetUser

fun main() {
    println(greetUser("  alice  "))
}
```

Concepts Used:

- Package declaration
- Single-name import
- private visibility (file-level)
- public by default (greetUser)
- Hard keywords: fun, return, val

Output: `Hello, ALICE`

Exercise 2: Class Visibility

Create a BankAccount class with private balance, protected deposit method, and a public getBalance function. Show inheritance.

Solution

```
open class BankAccount(
    private var balance: Double = 0.0
) {
    protected open fun deposit(amount: Double) {
        balance += amount
    }
    fun getBalance() = balance
}

class SavingsAccount : BankAccount() {
    fun addSavings(amount: Double) {
        deposit(amount) // OK: protected
    }
}

fun main() {
    val acct = SavingsAccount()
    acct.addSavings(100.0)
    println(acct.getBalance()) // 100.0
}
```

Concepts Used:

- private visibility (class member)
- protected visibility (subclass access)
- public by default (getBalance)
- open/override keywords
- Inheritance with class hierarchy

Output: 100.0

Exercise 3: Aliased Imports

Resolve a name clash between two classes named "Logger" from different packages using aliased imports.

Solution

```
import com.app.logging.Logger
import com.lib.debug.Logger as DebugLogger

fun main() {
    val appLog = Logger()
    val dbgLog = DebugLogger()

    appLog.log("App started")
    dbgLog.log("Debug info")
}
```

Concepts Used:

- Renaming import (as keyword)
- Resolving name clashes
- Single import vs aliased import
- Soft keyword: as

Your Turn — Practice Exercises

1 Keyword Detective

Which of these can be used as a variable name: val, by, class, get, for, constructor? Why?

Hint: Distinguish hard vs soft keywords.

2 Star vs Single

Rewrite star imports as single imports. Which approach is preferred and why?

Hint: Think about overload resolution.

3 Visibility Puzzle

Given a class with private, protected, and internal members, predict which are accessible from a subclass and an unrelated class.

Hint: Draw a visibility table.

4 Module Boundaries

Explain what happens when you try to access an internal class from a different module. How does internal differ from public?

Hint: Think Gradle modules.

Challenge Exercises

5 Sealed Class Visibility

Create a sealed class hierarchy with internal visibility. Add a when expression that pattern-matches all subtypes. What happens if you add a new subtype in a different module?

Hint: sealed + internal enforces module-level exhaustiveness.

6 Multi-Module Design

Design a library module exposing only a public API facade. Use internal for implementation classes and private for helpers. Show how import statements change for consumers of your library.

Hint: Combine packages, imports, and visibility for clean APIs.

Tips for Success

- Experiment in the Kotlin Playground
- Check visibility rules with the compiler
- Read the official docs for edge cases
- Try breaking visibility constraints on purpose
- Test how renaming affects imports
- Practice multi-file Kotlin projects